



ChefStack

Guía técnica del backend

Deep-dive de la API REST: arquitectura por capas, modelo de datos, endpoints y seguridad con Firebase. Spring Boot 3.3.5 · Java 21 · MySQL.

1 Stack y dependencias

El backend de ChefStack es una API REST construida sobre el ecosistema Spring Boot, siguiendo una arquitectura por capas con inversión de dependencias (interfaz + implementación en la capa de servicio). Persiste en MySQL mediante Spring Data JPA/Hibernate, valida sesiones con tokens de Firebase y enriquece las recetas con datos nutricionales de Spoonacular.

Tecnología	Versión / detalle	Rol en el proyecto
Spring Boot	3.3.5	Framework base: autoconfiguración, servidor embebido, inyección de dependencias.
Java	21 (LTS)	Lenguaje y runtime; uso de <code>record</code> para DTOs y <code>ApiError</code> .
Maven	<code>pom.xml</code>	Gestión de dependencias y empaquetado del artefacto.
Spring Web (MVC)	starter-web	Controladores REST, negociación de contenido, multipart.
Spring Data JPA	starter-data-jpa	Repositorios, mapeo ORM y consultas derivadas.
Hibernate	Proveedor JPA	ORM; <code>ddl-auto=update</code> crea/actualiza las tablas.
MySQL	Driver mysql-connector-j	Base de datos relacional.
Spring Security	starter-security	Cadena de filtros, sesión stateless, reglas de acceso.
com.auth0:java-jwt	JWT	Verificación de firma RS256 del token de Firebase.
Lombok	@Getter, @Builder, @Setter...	Reduce boilerplate en entidades y componentes.
springdoc-openapi	Swagger UI	Documentación interactiva de la API en <code>/swagger-ui</code> .
RestTemplate	Spring Web	Cliente HTTP hacia Spoonacular (con timeouts).

Empaquetado y despliegue: el repositorio incluye `Dockerfile`, `system.properties` y `Procfile`, lo que permite ejecutar el JAR de forma local o en plataformas tipo PaaS. La configuración sensible (API keys, credenciales de BD) se externaliza vía `config/secrets.properties.example` y perfiles de Spring.

2 Estructura de carpetas

El proyecto sigue el empaquetado por capa (*package-by-layer*) bajo el paquete raíz `com.chefstack.api`. Cada carpeta corresponde a una responsabilidad técnica clara:

```
chefstack-api/
├─ pom.xml, Dockerfile, system.properties, Procfile      # build, contenedor y arranque PaaS
├─ config/secrets.properties.example                    # plantilla de secretos (no versionar reales)
└─ src/main/
   └─ resources/                                         # configuración por perfil
      ├── application.properties                         # base / común
      ├── application-dev.properties                    # MySQL local
      └── application-prod.properties                   # variables de entorno
   └─ java/com/chefstack/api/
      ├── ChefstackApiApplication.java                 # punto de entrada @SpringBootApplication
      ├── model/                                         # Entidades JPA: Categoria, Receta, Ingrediente, Favorito, ImagenSubida, Dificultad
      ├── dto/                                           # Contratos de E/S: *Request / *Response, CategoriaResumenResponse, FavoritoResponse
      ├── mapper/                                         # Conversión entidad↔DTO: CategoriaMapper, RecetaMapper, IngredienteMapper
      ├── repository/                                     # Acceso a datos (Spring Data JPA): 5 repositorios
      ├── service/                                       # Lógica de negocio: interfaz + impl/ (DIP)
      ├── controller/                                   # Endpoints REST: Categoria, Receta, Ingrediente, Favorito, Upload
      ├── security/                                       # Firebase: verificador, filtro, SecurityConfig, entry point 401
      ├── client/                                        # Integraciones externas: NutricionClient, SpoonacularClient, dto/
      ├── exception/                                    # Errores: GlobalExceptionHandler, excepciones propias, ApiError
      └─ config/                                         # Infra transversal: OpenApiConfig, RestTemplateConfig, DataSeeder
```

Lectura rápida del flujo: una petición entra por `controller/` → atraviesa la cadena de `security/` → se delega a `service/` (reglas de negocio) → que usa `repository/` (BD) y, si aplica, `client/` (Spoonacular). Los `mapper/` traducen entre `model/` y `dto/`, y cualquier error se centraliza en `exception/`.

3 Modelo de datos

Cinco entidades JPA en `model/` definen el esquema relacional. Hibernate genera las tablas automáticamente (`ddl-auto=update`). A continuación, los campos de cada entidad.

Categoria

Campo	Tipo	Notas
id	Long	PK autogenerada.
nombre	String	Nombre de la categoría.
descripcion	String	Texto descriptivo.
recetas	List<Receta>	@OneToMany (lado inverso) hacia Receta.

Receta

Campo	Tipo	Notas
id	Long	PK autogenerada.
nombre	String	Título de la receta.
descripcion	String	Resumen.
instrucciones	String (<code>TEXT</code>)	Pasos de preparación, columna larga.
tiempoPreparacion	Integer	Minutos estimados.
dificultad	Dificultad (enum)	<code>FACIL</code> / <code>MEDIA</code> / <code>DIFICIL</code> .
porciones	Integer	Número de raciones.
imagenUrl	String	URL de la imagen (p. ej. <code>/api/uploads/{id}</code>).
calorias	Integer	Estimado por Spoonacular al crear.
categoria	Categoria	@ManyToOne (FK <code>categoria_id</code>).
ingredientes	List<Ingrediente>	@OneToMany con <code>cascade=ALL</code> + <code>orphanRemoval=true</code> .

Ingrediente

Campo	Tipo	Notas
id	Long	PK autogenerada.
nombre	String	Nombre del ingrediente.
cantidad	Double	Cantidad numérica.
unidad	String	Unidad de medida (g, ml, taza...).
receta	Receta	@ManyToOne (FK <code>receta_id</code>).

Favorito

Campo	Tipo	Notas
id	Long	PK autogenerada.
firebaseUid	String	UID del usuario autenticado (Firebase).
receta	Receta	@ManyToOne (FK <code>receta_id</code>).
creadoEn	Instant / LocalDateTime	Marca de tiempo de creación.

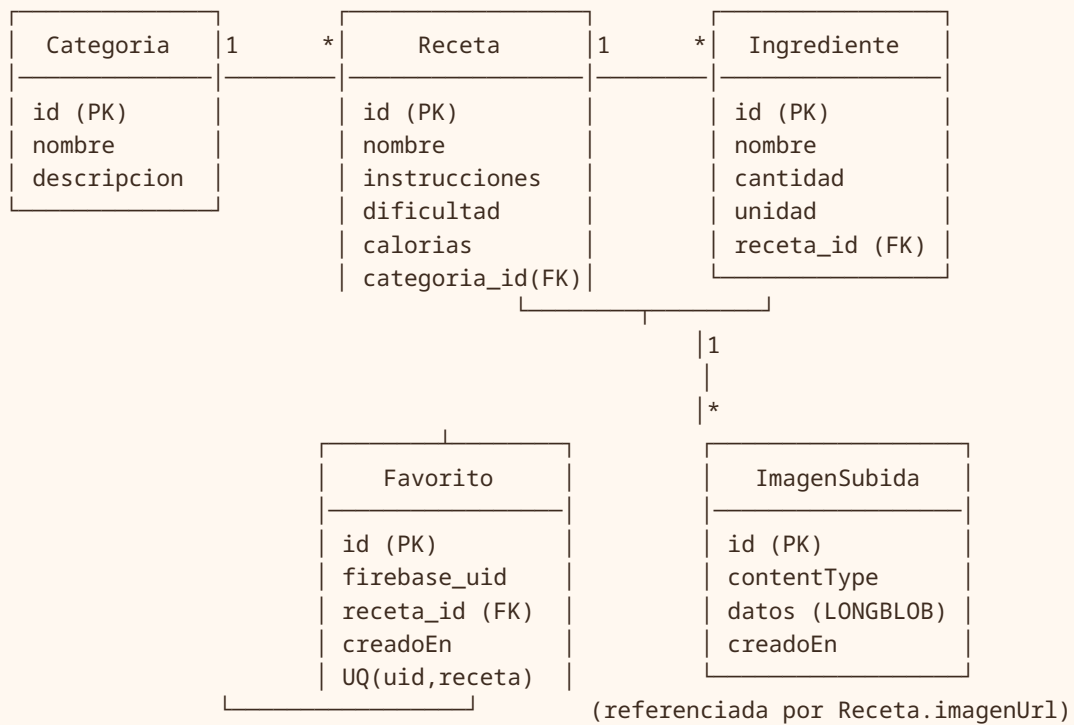
Restricción única sobre `(firebase_uid, receta_id)`: un usuario no puede marcar dos veces la misma receta.

ImagenSubida

Campo	Tipo	Notas
id	Long	PK autogenerada; forma la URL pública.
contentType	String	MIME (p. ej. <code>image/png</code>).
datos	byte[] (<code>LONGBLOB</code>)	Bytes binarios del archivo.
creadoEn	Instant / LocalDateTime	Fecha de subida.

Relaciones

El núcleo del dominio es la cadena **Categoría 1:N Receta 1:N Ingrediente**. **Favorito** asocia un UID de Firebase con una receta (N:1), y **ImagenSubida** es independiente: almacena los binarios que luego se referencian desde `Receta.imageUrl`.



Integridad referencial: al eliminar una receta hay que respetar la FK de **Favorito** . Por eso el servicio borra primero los favoritos asociados y luego la receta (ver §4). Los ingredientes, en cambio, se eliminan en cascada vía **orphanRemoval** .

4 Capa por capa

model/ — Entidades de dominio

Clases JPA anotadas con **@Entity** y Lombok. Definen tablas, columnas y relaciones descritas en §3. **Dificultad** es un **enum** (**FACIL** , **MEDIA** , **DIFICIL**) persistido en la columna correspondiente de **Receta** . La cascada y el **orphanRemoval** de **Receta → Ingrediente** permiten gestionar los ingredientes anidados como un agregado.

repository/ — Acceso a datos

Interfaces que extienden **JpaRepository** ; Spring Data genera la implementación. Aportan CRUD y consultas derivadas por nombre de método:

Repositorio	Responsabilidad / consultas típicas
CategoriaRepository	CRUD de categorías.
RecetaRepository	CRUD; filtro por categoría y búsqueda por nombre (<code>findByCategoriaId</code> , <code>findByNombreContaining...</code>).
IngredienteRepository	CRUD; <code>findByRecetaId</code> .
FavoritoRepository	Consultas por <code>firebaseUid</code> ; existencia/borrado por (<code>uid</code> , <code>recetaId</code>) ; borrado por <code>recetaId</code> .
ImagenRepository	Persistir y recuperar binarios de imagen por id.

dto/ — Contratos de entrada/salida

Separan el modelo interno del contrato HTTP. Los `*Request` llevan validaciones (Bean Validation) y los `*Response` moldean lo que ve el cliente:

- `CategoriaRequest` / `CategoriaResponse` / `CategoriaResumenResponse` (versión reducida embebida en la receta).
- `RecetaRequest` / `RecetaResponse` .
- `IngredienteRequest` / `IngredienteResponse` / `IngredienteEnRecetaRequest` (ingrediente anidado al crear receta).
- `FavoritoResponse` .

mapper/ — Conversión entidad↔DTO

Componentes sin estado que traducen en ambos sentidos. `RecetaMapper` arma el `RecetaResponse` incluyendo la `CategoriaResumenResponse` y la lista de `IngredienteResponse` ; convierte `RecetaRequest` + ingredientes anidados en la entidad `Receta` . `CategoriaMapper` e `IngredienteMapper` hacen lo propio para sus dominios.

service/ — Lógica de negocio (interfaz + impl)

Cada servicio se expone como interfaz y se implementa en `impl/` , aplicando el principio de inversión de dependencias: los controladores dependen de la abstracción, no de la implementación concreta.

RecetaService / RecetaServiceImpl

Método	Comportamiento
listar(categoriaId?, buscar?)	Lista todas las recetas o filtra por categoría y/o término de búsqueda.
obtener(id)	Devuelve una receta; lanza <code>ResourceNotFoundException</code> si no existe.
crear(request)	Arma la entidad, agrega los ingredientes anidados, invoca <code>NutricionClient.estimarCalorias(nombre)</code> para fijar las calorías y persiste todo el agregado.
actualizar(id, request)	Sustituye campos y la lista de ingredientes (gestionada por <code>orphanRemoval</code>).
eliminar(id)	Regla: primero borra los favoritos que apuntan a esa receta (evita violar la FK) y luego elimina la receta.

CategoriaService / CategoriaServiceImpl

Método	Comportamiento
listar() / obtener(id)	Lectura; 404 si no existe.
crear / actualizar	Alta y edición de categorías.
eliminar(id)	Regla: no permite borrar una categoría que aún tiene recetas asociadas → <code>BadRequestException</code> (400).

IngredienteService / IngredienteServiceImpl

CRUD de ingredientes, normalmente filtrados por `recetaId`. Permite gestionar ingredientes de forma independiente además del flujo anidado de la receta.

FavoritoService / FavoritoServiceImpl

Método	Comportamiento
listarPorUid(uid)	Devuelve los favoritos del usuario autenticado.
agregar(uid, recetaId)	Idempotente: si ya existe el par <code>(uid, receta)</code> no duplica; respeta la restricción única.
eliminar(uid, recetaId)	Quita la receta de los favoritos del usuario.

controller/ — Endpoints REST

Exponen los servicios vía HTTP, validan los `*Request` y delegan en la capa de servicio.

`CategoriaController`, `RecetaController`, `IngredienteController` y `FavoritoController` cubren el CRUD del dominio; `UploadController` gestiona la subida y entrega de imágenes (multipart). El detalle de rutas está en §5.

security/ — Autenticación con Firebase

Cuatro piezas implementan la seguridad (detalle en §6): `FirebaseTokenVerifier` (valida el JWT contra las llaves públicas de Google), `FirebaseAuthenticationFilter` (`OncePerRequestFilter` que intercepta el header `Authorization`), `SecurityConfig` (cadena de filtros, CORS, sesión stateless y reglas de acceso) y `RestAuthenticationEntryPoint` (respuesta 401 en JSON).

client/ — Integraciones externas

`NutricionClient` es la abstracción de estimación nutricional; `SpoonacularClient implements NutricionClient` la implementa usando `RestTemplate` contra el endpoint `guessNutrition` de Spoonacular, mapeando la respuesta con `dto/GuessNutritionResponse`. Si falta la API key o la llamada falla, devuelve `Optional` vacío para **no romper la creación de la receta**.

exception/ — Manejo centralizado de errores

`GlobalExceptionHandler` (`@RestControllerAdvice`) traduce excepciones a respuestas HTTP consistentes usando el `record ApiError`. Excepciones propias: `ResourceNotFoundException` (404) y `BadRequestException` (400). Detalle en §7.

config/ — Infraestructura transversal

- `OpenApiConfig`: configura Swagger/OpenAPI y declara el esquema de seguridad *bearer* (token de Firebase).
- `RestTemplateConfig`: expone el `RestTemplate` con timeouts de conexión y lectura para el cliente de Spoonacular.
- `DataSeeder`: al arrancar, si la BD está vacía, siembra 10 recetas demo distribuidas en 5 categorías.

5 Endpoints de la API

Prefijo base `/api`. La columna *Auth* indica si la ruta es pública o requiere token de Firebase. Como regla general: las lecturas (`GET`) del catálogo son públicas; las escrituras y todo lo de favoritos/uploads son privadas.

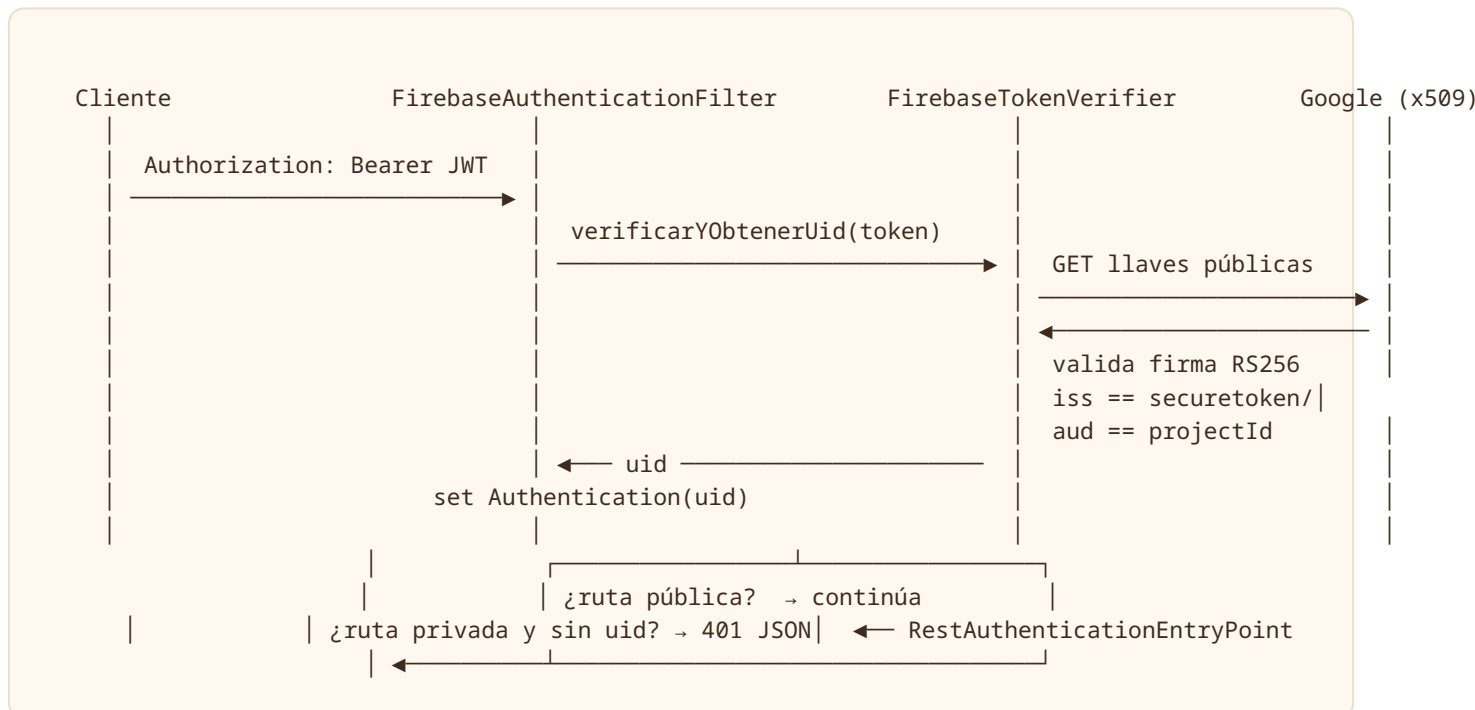
Método	Ruta	Auth	Descripción
GET	/api/categorias	Público	Lista todas las categorías.
GET	/api/categorias/{id}	Público	Detalle de una categoría.
POST	/api/categorias	Privado	Crea una categoría.
PUT	/api/categorias/{id}	Privado	Actualiza una categoría.
DELETE	/api/categorias/{id}	Privado	Elimina (400 si tiene recetas asociadas).
GET	/api/recetas? categoriaId=&buscar=	Público	Lista recetas; filtra por categoría y término de búsqueda.
GET	/api/recetas/{id}	Público	Detalle con categoría resumida e ingredientes.
POST	/api/recetas	Privado	Crea receta + ingredientes; estima calorías vía Spoonacular.
PUT	/api/recetas/{id}	Privado	Actualiza receta e ingredientes.
DELETE	/api/recetas/{id}	Privado	Elimina (borra antes sus favoritos).
GET	/api/ingredientes?recetaId=	Público	Lista ingredientes, opcionalmente por receta.
POST	/api/ingredientes	Privado	Crea un ingrediente.
PUT	/api/ingredientes/{id}	Privado	Actualiza un ingrediente.
DELETE	/api/ingredientes/{id}	Privado	Elimina un ingrediente.
GET	/api/favoritos	Privado	Favoritos del usuario autenticado (por UID).
POST	/api/favoritos/{recetaId}	Privado	Marca como favorita (idempotente).
DELETE	/api/favoritos/{recetaId}	Privado	Quita de favoritos.
POST	/api/uploads	Privado	Sube imagen (multipart, máx 5MB); devuelve {url: "/api/uploads/{id}"} .
GET	/api/uploads/{id}	Público	Sirve los bytes de la imagen almacenada.
GET	/swagger-ui , /v3/api-docs	Público	Documentación interactiva OpenAPI.

6 Seguridad en detalle

La autenticación delega 100% en Firebase Authentication. El backend **no gestiona contraseñas**: el cliente obtiene un ID token de Firebase y lo envía en cada petición. La sesión es *stateless* (sin cookie de servidor).

Flujo del token, paso a paso

1. El cliente añade el header `Authorization: Bearer <ID_TOKEN>`.
2. `FirebaseAuthenticationFilter` (un `OncePerRequestFilter`) intercepta la petición y extrae el token del header.
3. Llama a `FirebaseTokenVerifier.verificarYObtenerUid(token)`.
4. El verificador descarga las **llaves públicas** de Google (endpoint x509 de `securetoken`) y valida la **firma RS256**.
5. Comprueba el **emisor** (`https://securetoken.google.com/<projectId>`) y la **audiencia** (`projectId`).
6. Si todo es válido, coloca el `uid` como *principal* autenticado en el `SecurityContext`.
7. Las reglas de `SecurityConfig` deciden si la ruta requiere autenticación.
8. Si la ruta es privada y no hay sesión válida, `RestAuthenticationEntryPoint` responde **401** en JSON.



Configuración de SecurityConfig

- Sesión **stateless** (`SessionCreationPolicy.STATELESS`); CSRF deshabilitado para API.
- **CORS** habilitado para el frontend.
- Swagger y `/v3/api-docs` públicos.
- El filtro de Firebase se inserta antes del filtro estándar de autenticación.

Reglas de acceso por ruta/método

Patrón de ruta	Método	Acceso
<code>/api/categorias/**</code> , <code>/api/recetas/**</code> , <code>/api/ingredientes/**</code>	GET	Público
Los mismos recursos	POST / PUT / DELETE	Autenticado
<code>/api/favoritos/**</code>	Todos	Autenticado
<code>/api/uploads</code>	POST	Autenticado
<code>/api/uploads/{id}</code>	GET	Público
<code>/swagger-ui/**</code> , <code>/v3/api-docs/**</code>	GET	Público

Identidad del usuario: el `uid` de Firebase actúa como identificador de propietario en `Favorito.firebaseUid`. Los servicios de favoritos siempre operan con el UID del principal autenticado, evitando que un usuario vea o modifique los favoritos de otro.

7 Manejo de errores

`GlobalExceptionHandler` (`@RestControllerAdvice`) centraliza la traducción de excepciones a respuestas HTTP uniformes. Todas las respuestas de error usan el mismo formato: el `record ApiError`.

Excepción	Código HTTP	Cuándo se produce
<code>ResourceNotFoundException</code>	404 Not Found	Entidad inexistente (id no encontrado).
<code>BadRequestException</code>	400 Bad Request	Violación de regla de negocio (p. ej. borrar categoría con recetas).
<code>MethodArgumentNotValidException</code>	400 Bad Request	Falla de validación de un <code>*Request</code> ; rellena <code>erroresCampos</code> por campo.
<code>Exception</code> (genérica)	500 Internal Server Error	Cualquier error no controlado.

Estructura del record `ApiError`

Campos: `timestamp`, `status`, `error`, `mensaje`, `ruta` y `erroresCampos` (mapa campo→mensaje, presente solo en errores de validación).

```
{
  "timestamp": "2026-06-28T14:32:10.482Z",
  "status": 400,
  "error": "Bad Request",
  "mensaje": "Datos de entrada inválidos",
  "ruta": "/api/recetas",
  "erroresCampos": {
    "nombre": "no debe estar vacío",
    "porciones": "debe ser mayor que 0"
  }
}
```

Resiliencia ante terceros: la integración con Spoonacular está aislada del flujo de error. Si la estimación de calorías falla, el `SpoonacularClient` devuelve `Optional` vacío y la receta se crea igualmente sin calorías, en lugar de propagar un 500 al cliente.

Perfiles de configuración

Perfil	Archivo	Uso
base	<code>application.properties</code>	Configuración común (JPA, OpenAPI, etc.).
dev	<code>application-dev.properties</code>	MySQL local; ideal para desarrollo.
prod	<code>application-prod.properties</code>	Credenciales y endpoints vía variables de entorno.

En todos los perfiles, Hibernate con `ddl-auto=update` crea o actualiza el esquema, y `DataSeeder` garantiza datos demo en una BD vacía.